

UPSPA IMPLEMENTATION NOTES

Operations Runbook No. 1 • companion to Implementation Reports I (May 2026) and II (June 2026)

UpSPA Browser Extension

Deployment, Demo, and Troubleshooting Runbook

Operational companion to the Tasks 3 & 6 implementation report

Efe Bekteş

Istanbul Technical University, Department of Computer Engineering
Summer intern, CRYPTO@KU, Koç University • bekteş23@itu.edu.tr
June 2026 • github.com/rezasaadi/UpSPA_extension

Summary. This runbook records, for the next person, exactly how to bring up the UpSPA demo on a Windows host with WSL2 and exercise the full register/login flow through the browser extension, and it catalogues every problem encountered during that bring-up together with its root cause and fix. The intended audience is a teammate reproducing the demo for the first time; the operational details below are deliberately concrete because most of them are not obvious from the source. Each failure is shown with the actual on-screen message captured during the session, followed by why it happens and how it is resolved. The accompanying implementation report [2] covers the Task 3 (password-policy detection) and Task 6 (deterministic encoder) design and tests; here the focus is deployment and the debugging path to a green end-to-end run. None of the issues below were defects in the assigned client logic; they were environment, provisioning, and operator-sequence problems, but documenting them saves the next person the hour they cost here. It is the first of a planned series of operational runbooks and a companion to the two implementation reports: Report I [1] (Storage-Provider cryptography and deployment) and Report II [2] (this extension's policy detection and deterministic encoding).

1 Environment and architecture

The backend (Storage Providers, login servers, PostgreSQL) and the build toolchain run inside **WSL2 (Ubuntu)** on a Windows 11 host; **Chrome runs natively on Windows** and loads the extension. This split is forced by the demo scripts, which are Linux shell and use `tmux` and `pg_isready`; they even print a second set of Storage-Provider URLs on the WSL IP specifically “for Chrome on Windows.” The cross-boundary reachability that this implies is the source of several issues in §5.

Table 1: Components, where they run, and their ports.

Component	Runtime	Port(s)	Notes
Login servers (×3)	Node (WSL)	3000, 3001, 3002	<code>demo/light-login-server</code> ; the demo uses <code>localhost:3000</code> .
Storage Providers (×3)	Go (WSL)	8081, 8082, 8083	<code>services/storage-provider-go</code> ; threshold 2-of-3.
PostgreSQL	WSL service	5432	Role <code>reza</code> ; databases <code>upspa1/2/3</code> .
WASM core + client	built in WSL	n/a	<code>crates/upspa-wasm</code> → <code>packages/upspa-js</code> .
Extension	Chrome (Windows)	n/a	loaded unpacked from <code>packages/extension/dist</code> .

The per-site password the extension fills is bound to the *login-server domain*; the Storage-Provider URLs do not affect

it. Practically, this means **registration and login must both happen on** `localhost:3000`, but the SP list may be either the `localhost` or the WSL-IP addresses, whichever Chrome can reach (§5.5).

2 One-time setup

The toolchains (Node 20, Go, Rust with `wasm-pack`, PostgreSQL), the PostgreSQL role and databases, and the build chain `WASM → JS → extension` are installed once by the bootstrap script. Two setup details are worth recording because they are not obvious.

wasm-pack cannot fetch the optimiser on a fresh host. On first build, `wasm-pack` tries to download the `binaryen/wasm-opt` optimiser from GitHub and fails on a constrained network, aborting the build. The fix is to disable the optimiser for the demo build by adding the following to `crates/upspa-wasm/Cargo.toml`; the emitted WebAssembly is otherwise identical and this is a local-only build setting (do not commit it).

Listing 1: Local build workaround appended to `crates/upspa-wasm/Cargo.toml`.

```
[package.metadata.wasm-pack.profile.release]
wasm-opt = false
```

The task branch. The Tasks 3 and 6 implementation is committed on the `intern/extension` branch (base `v0.1_local`); that is the branch on which the repository's guard hook permits `packages/extension/` commits. Check it out and rebuild the extension before loading the `dist` folder.

Loading the extension into Chrome. In `chrome://extensions`, enable *Developer mode*, click *Load unpacked*, and select the built `dist` folder. From Windows that folder is under the WSL share:

```
\\wsl.localhost\Ubuntu\home\<user>\upspa\UpSPA_extension\packages\extension\dist
```

After any rebuild, click the extension's reload icon (or remove and re-load) and hard-refresh the page with `Ctrl+Shift+R`. The popup markup is identical between builds, so a reload is the only way to be sure Chrome is running the current bundle.

3 Bring-up and reset cycle

A single pair of scripts stops everything, clears the three PostgreSQL databases, and restarts the Storage Providers and login servers under `tmux`. Run this whenever a clean slate is needed (and note §5.4: provisioning must be redone after every reset).

Listing 2: Reset and restart, run from the repository root in WSL.

```
bash reset_demo.sh
GO_BIN=/usr/local/go/bin/go bash start_demo.sh
```

The script prompts for the PostgreSQL password while truncating each database, then health-checks the services. A successful run ends with three `{"ok":true}` responses, the running `tmux` sessions, and both URL sets; the full transcript is in Appendix A. Immediately after a reset the login server shows a clean state (Fig. 1).

UpSPA

Current tab origin

http://localhost:3000

Known accounts

No accounts saved for this site

Add or edit account ID

alice

Add / update account

Delete selected account

Master password

.....

Detect password policy

Password policy

Min	Max
20	32

Figure 1: The light login server home page immediately after a reset: “No accounts registered yet.” The server only stores and verifies the password value the browser submits; it knows nothing about UpSPA.

4 The end-to-end recipe that works

The following sequence produces a green login. The two operator habits that matter most are folded in: (i) use one known master password everywhere, and (ii) do both popup actions *before* clicking on the page, because clicking the page closes the popup and clears its state.

1. **Reset** (§4), then in chrome://extensions reload the extension.
2. **Provision.** Open the extension *Options*: UID test-user, master password test-password, threshold 2, Storage Providers localhost:8081/8082/8083. Click *Setup / Provision SPs* (the second button, *not* “Save Config Only”) and wait for the success status. This step is mandatory and is the one most often skipped (§5.4).
3. **Register.** On localhost:3000/register, open the popup: account alice, master password test-password, click *Detect password policy* (Fig. 3b), then *Register current site* (Fig. 6a). Do not touch the page yet.
4. **Save the per-site record** by clicking *Add / update account* in the popup (Fig. 6b); login requires this record.
5. **Submit the page’s Register form.** Only now click into the page and press its *Register* button; the server reports the account exists (Fig. 7a).

6. **Login.** On `localhost:3000/login`, open the popup, select `alice` from *Known accounts*, type `test-password`, click *Login current site* (Fig. 8a), then submit the page's *Login* form.

Done correctly, the server confirms registration and then authentication (Figs. 2a–2b). The login value is re-derived deterministically from the same per-site secret, counter, and policy as registration, which is exactly the Task 6 property the demo is meant to show.

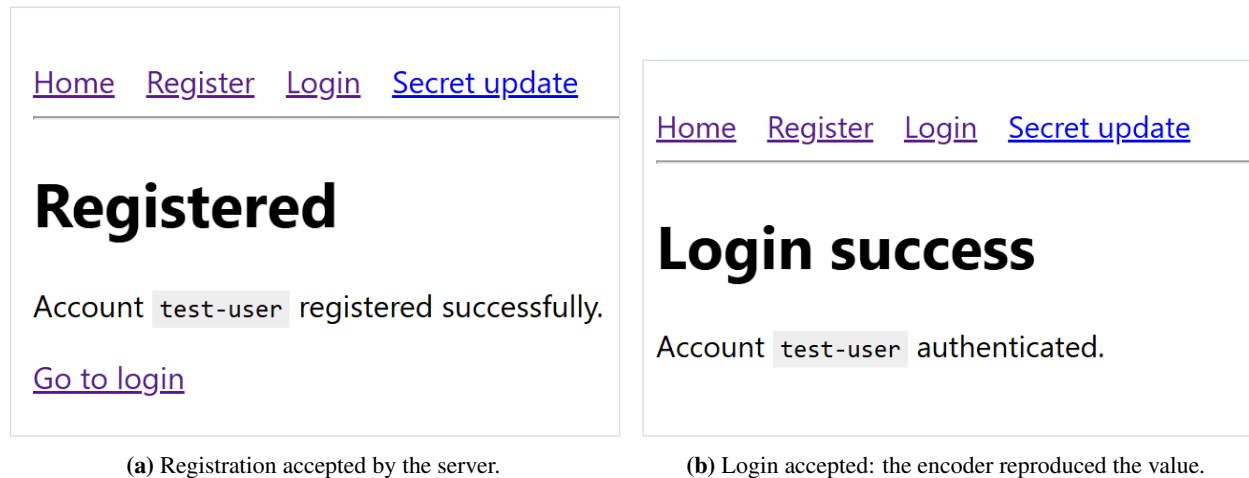


Figure 2: The successful end-to-end result for `test-user` on `localhost:3000`.

5 Troubleshooting catalogue

Each entry below is an issue actually hit during bring-up, with the on-screen message, the cause, and the resolution. They are ordered roughly as they surfaced.

5.1 Build: `wasm-pack` cannot download the optimiser

Symptom. The build aborts while `wasm-pack` fetches `binaryen/wasm-opt` from GitHub. **Cause.** Network restrictions on the build host block the optimiser download. **Fix.** Disable `wasm-opt` in the crate metadata (the listing in §3); the WebAssembly is otherwise identical. Local-only; not committed.

5.2 Detect: “No visible password field found for policy detection”

Symptom. Fig. 3a: clicking *Detect password policy* returns a red error and the evidence line reads “Policy detection failed; using safe defaults.” **Cause.** The popup scans the *active tab*, and the detection was run on the login server *home* page, which has no password form. **Fix.** Navigate to `/register` (or `/login`) first; the detector then finds the field. On the demo form, which states no rules, the detected policy is the safe default (Min 20, Max 32, all classes), and the evidence line confirms the field was located by its `autocomplete` attribute (Fig. 3b).

(a) Detect on a page with no form.

(b) Detect on the register form: evidence + “policy detected.”

Figure 3: Policy detection fails on a page with no password field (left) and succeeds on the register form (right), reporting `password input autocomplete=new-password` as its evidence.

5.3 “Master password is empty”

Symptom. Fig. 4: an action returns “Master password is empty.” even though it was typed a moment earlier. **Cause.** The popup intentionally clears the master-password field every time it closes (it is never stored). Reopening the popup, or clicking the page, empties it. **Fix.** Retype the master password immediately before each action (*Register current site*, *Login current site*). This is the single most frequent operator trip-up in the whole flow.

Figure 4: The master-password field clears on every popup close; retype it before each action.

5.4 “Registration: only 0/3 SP record writes succeeded (< threshold 2)”

Symptom. Fig. 5: registration reports that none of the three Storage-Provider record writes succeeded. **Cause.** The Storage Providers were not provisioned for this user, so `POST /v1/records` has no setup to attach the record to and is rejected by all three. On a fresh reset this is invariably because *Setup / Provision SPs* was not run (or “Save Config

Only” was clicked instead, which only saves local config). **Fix.** Open Options and click *Setup / Provision SPs*; confirm the success status, then register again. **Provisioning must be redone after every reset**, because the reset truncates the databases. If provisioning itself fails, the SPs are unreachable from Chrome; see §5.5.

The screenshot shows a vertical stack of buttons within a light gray border. At the top, the text "Counter 0." is displayed. The buttons, from top to bottom, are: "Register current site", "Generate another candidate", "Confirm Registration Success" (which is greyed out), "Login current site", "Prepare secret update", "Commit secret update after website success" (greyed out), "Cancel prepared secret update" (greyed out), a red-bordered box containing the text "Registration: only 0/3 SP record writes succeeded (< threshold 2).", and "Lock extension".

Figure 5: Registration before provisioning: 0/3 record writes succeed. The fix is to provision first.

5.5 Storage-Provider reachability: localhost vs. the WSL IP

Symptom. Provisioning or registration writes fail even though the page at localhost:3000 loads. **Cause.** After a WSL reset, the Windows-to-WSL localhost relay for the SP ports can be stale, so Chrome cannot reach localhost:8081-8083 for programmatic writes even though the navigated page works. **Fix.** Test both addresses from a Chrome tab, `http://localhost:8081/v1/health` and `http://172.17.138.203:8081/v1/health`, and put whichever returns `{"ok":...}` into the Options Storage-Provider list, then re-provision. (In this session both worked once the relay refreshed; the WSL-IP form is the robust fallback.) This is permission-safe: the manifest grants `host_permissions: ['<all_urls>']` and the SPs send `Access-Control-Allow-Origin: *`, so neither address is blocked by CORS (§6).

5.6 “Confirm Registration Success” is greyed out

Symptom. After submitting the page form, *Confirm Registration Success* is disabled on reopening the popup. **Cause.** The pending-registration state lives in the popup, not in storage; clicking the page closes the popup and discards it. **Fix.** Either click *Confirm Registration Success* before touching the page, or, as in the recipe, save the per-site record with *Add / update account* (Fig. 6b), which persists the policy and counter independently of the pending state.

5.7 Re-clicking “Register current site” → 0/3 again

Symptom. A second *Register current site* on an already-registered account returns the 0/3 error again. **Cause.** The first registration already wrote the per-site record to the SPs; a second attempt is a duplicate `POST /v1/records` and is rejected. This is a conflict, not a real failure. **Fix.** Do not re-register; the record exists. Use *Add / update account* to save the local mapping, and proceed to login.

(a) “Registration value filled” (2 password fields).

(b) “Account saved”, the per-site record persisted.

Figure 6: The registration value is filled by the popup (left); *Add / update account* then saves the local per-site record (right) that login depends on.

(a) Server: “Account already exists: alice.”

(b) Popup policy panel after detection (Counter 0).

Figure 7: After submitting the page form the server reports the account exists (left); the popup’s policy panel shows the detected values and the allowed-symbol set `!@#$$%^&*` (right).

5.8 “Login failed”

Symptom. Fig. 8b: the value is filled (Fig. 8a) but the server rejects the login. **Cause.** The light server compares the submitted login value byte-for-byte against the registered value. Registration submits `register(...).to_ls.vinfo`; login submits `authenticate(...).vinfo_prime($6)`. These agree only when the *same master password* is used at registration and login. Because the field clears every time, a one-character difference between the two had crept in, producing a different `vinfo_prime` and a rejected login. **Fix.** Use one consistent, simple master password (`test-password`) at provision, register, and login.

Home Register Login

Login

User enters account id and UpSPA master password. Extension replaces password with vInfoPrime.

Main UpSPA UID
alice

Login-server UID
alice

UpSPA master password

Counter 0.

Register current site

Generate another candidate

Confirm Registration Success

Login current site

Prepare secret update

Commit secret update after website success

Cancel prepared secret update

Login value filled (username filled, password fields filled: 1). Submit the website form manually.

Lock extension

Open Options

(a) “Login value filled” (1 password field).

Home Register Login Secret update

Login

User enters account id and UpSPA master password. Extension replaces password with vInfoPrime.

Login failed.

Main UpSPA UID

Login-server UID
alice

UpSPA master password

Login

(b) Server: “Login failed.”

Figure 8: Login value filled by the popup (left) but rejected by the server (right) when the login-time master password differs from the registration-time one.

5.9 “aead error”

Symptom. Fig. 9: login throws aead error. **Cause.** This is the decisive confirmation of the previous point. The per-site record is stored on the SPs encrypted under a key derived from the master password used *at registration*; at login, authenticate derives a key from the password typed now and tries to decrypt it. aead error is an AEAD authentication-tag failure, the login password does not match the one the record was created with. Changing only the login password cannot fix this, because the stored record is locked to the old password. **Fix.** Clear the old record (reset, §4) and *re-register* the account under the single known password, then log in with the same password. This is precisely why the recipe re-registers after a reset rather than only re-typing the login password.

comma separated

Counter 0.

Register current site

Generate another candidate

Confirm Registration Success

Login current site

Prepare secret update

Commit secret update after website success

Cancel prepared secret update

aead error

Lock extension

Open Options

Figure 9: aead error at login: the record is encrypted under the registration-time password; re-register under one consistent password to resolve it.

6 How the pieces fit (relevant code)

The behaviour behind the login failures is explained by a handful of excerpts. Registration and login derive the per-site value through *different* client calls, and they agree only by construction.

Listing 3: `packages/extension/src/popup/popup.ts`, register vs. login fill the value via different calls.

```
// register handler
const rawPasswordForLs = await registerForSite(lsj, masterPassword);
const encoded = await encodeSecretAsPassword(rawPasswordForLs, policy, accountId, currentEncoderCounter);
// ... UPSPA_FILL_REGISTER with encoded.password

// login handler
const account = await getAccountForOrigin(activeOrigin, accountId); // saved policy + counter
const rawPasswordForLs = await authenticateForSite(lsj, masterPassword);
const encoded = await encodeSecretAsPassword(rawPasswordForLs, account.passwordPolicy, accountId, account.encoderCounter);
// ... UPSPA_FILL_LOGIN with encoded.password
```

Listing 4: `src/shared/upspaActions.ts`, registration uses `vinfo`, login uses `vinfo_prime`.

```
export async function registerForSite(lsj, password, uid?) {
  const client = await makeUpspaClient(uid);
  return (await client.register(lsj, password)).to_ls.vinfo;
}
export async function authenticateForSite(lsj, password, uid?) {
  const client = await makeUpspaClient(uid);
  return (await client.authenticate(lsj, password)).vinfo_prime;
}
```

The core guarantees these agree for the same inputs; its own integration test asserts it, so the encoder (which is downstream and received identical counter/policy/account both times) is not the cause of a mismatch; a different *password* is.

Listing 5: `crates/upspa-core/tests/integration_flows.rs`, the invariant that makes login work.

```
let vinfo_reg = reg.to_ls.vinfo;
// ... authenticate with the same password ...
assert_eq!(auth_res.vinfo_prime, vinfo_reg);
```

Finally, the reachability fix in §5.5 is permission-safe because the extension may call any origin and the Storage Providers allow any caller:

Listing 6: Manifest permission and Storage-Provider CORS (both fully open for the demo).

```
// packages/extension/src/manifest.ts
host_permissions: ['<all_urls>']
// services/storage-provider-go/internal/api/middleware.go (withCORS)
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
```

7 Error-prone areas at a glance

Table 2 collects the traps, in the order they tend to bite, with the one-line remedy for each.

Table 2: The recurring trip-ups and their fixes.

Trap	Remedy
Stale Chrome bundle after a rebuild	Reload the extension in <code>chrome://extensions</code> ; <code>Ctrl+Shift+R</code> the page.
Detect run on a page with no form	Be on <code>/register</code> or <code>/login</code> before detecting.
Master-password field clears on close	Retype it immediately before each Register/Login action.
Skipping provisioning after a reset	Click <i>Setup / Provision SPs</i> (not “Save Config Only”); confirm success.
SP unreachable from Chrome	Use whichever of <code>localhost</code> / WSL-IP returns <code>/v1/health</code> ; re-provision.
Clicking the page closes the popup	Do <i>Register + Add/update account</i> (or <i>Confirm</i>) before touching the page.
Re-registering an existing account	Don’t; the record exists. Save the local mapping and log in.
Different master password at login	Use one known password everywhere; otherwise login fails / <code>aead error</code> .
Old record locked to old password	Reset and re-register under the single known password.
Wrong login-server host	Register and login both on <code>localhost:3000</code> (the value is domain-bound).

Scope note. Every item above is an environment, provisioning, or operator-sequence matter. The assigned Task 3 detector and Task 6 encoder behaved correctly throughout: detection located the field and produced the policy (Fig. 3b), and once a single consistent master password was used, the encoder reproduced the per-site value and login succeeded (Fig. 2). Two of these traps also point at natural follow-up work that is not in the assigned scope: the popup could persist (or at least not silently clear) the master password within a session, and registration could detect an existing record and offer to update rather than fail on a duplicate write.

References

- [1] E. Bekteř. *Updatable Single Password Authentication: deployment, Storage-Provider cryptography, and a browser-free demo*. UpSPA Implementation Report I, CRYPTO@KU internship, May 2026.
- [2] E. Bekteř. *Updatable Single Password Authentication: client-side policy detection and deterministic password derivation in the browser extension*. UpSPA Implementation Report II, CRYPTO@KU internship, June 2026.

A Full bring-up transcript

The reset-and-restart transcript from the session, verbatim.

Listing 7: `reset_demo.sh` followed by `start_demo.sh`.

```
$ bash reset_demo.sh
GO_BIN=/usr/local/go/bin/go bash start_demo.sh
[1/6] Stopping SP servers...
[2/6] Stopping login servers...
[3/6] Clearing PostgreSQL databases...
Resetting database: upspa1
Password for user reza:
TRUNCATE TABLE
TRUNCATE TABLE
Resetting database: upspa2
Password for user reza:
TRUNCATE TABLE
TRUNCATE TABLE
Resetting database: upspa3
Password for user reza:
TRUNCATE TABLE
TRUNCATE TABLE
[4/6] Removing extension test state (manual step)
Remove/reload extension from chrome://extensions
[5/6] Checking ports...
OK: Port 3000 is free
OK: Port 3001 is free
OK: Port 3002 is free
OK: Port 8081 is free
OK: Port 8082 is free
OK: Port 8083 is free
[6/6] Demo state fully reset.
[1/5] Starting SP1...
[2/5] Starting SP2...
[3/5] Starting SP3...
```

```
[4/5] Starting Login Server 1...
[5/5] Starting Login Server 2...
[6/5] Starting Login Server 3...
Checking health...
{"ok":true}
{"ok":true}
{"ok":true}
Running tmux sessions:
ls1: 1 windows  ls2: 1 windows  ls3: 1 windows
sp1: 1 windows  sp2: 1 windows  sp3: 1 windows
Login servers:
  http://localhost:3000
  http://localhost:3001
  http://localhost:3002
Storage Providers:
  http://localhost:8081
  http://localhost:8082
  http://localhost:8083
Storage Providers for Chrome on Windows:
  http://172.17.138.203:8081
  http://172.17.138.203:8082
  http://172.17.138.203:8083
Done.
```